

RTO Risk Scorer — Product Requirements Document

Version: 1.0
Date: 2026-08-26
Author: Buildathon Submission Team
Status: Draft → Ready for Development
Track: Razorpay AI Buildathon — Track 02: AI Risk Manager
Deadline: 2026-09-05 (10 days)

1. Executive Summary

1.1 Problem Statement

In Indian e-commerce, **Return-to-Origin (RTO)** rates for Cash-on-Delivery (COD) orders range from **25–40%**. Each RTO costs the merchant **50–150** in logistics, packaging, and opportunity cost. Small merchants lack the tools to identify which orders will RTO before they ship, leading to preventable losses.

1.2 Solution

RTO Risk Scorer is a multi-agent AI pipeline that evaluates every COD order at checkout-time and assigns a **0–100 risk score** with a deterministic recommendation:

Score	Recommendation	Action
0–25	Auto-Approve	Ship immediately
26–60	Manual Review	Flag for verification call or partial prepayment
61–100	Auto-Reject	Cancel or request 100% prepayment

1.3 Core Philosophy

“Deterministic decisions, LLM explanations, graceful degradation.”

- Risk scores are computed by **pure Python rules** — auditable, reproducible, no black box.
- LLM generates **human-readable narratives** only — never overrides the score.
- If any agent fails, the system **degrades gracefully** with a lower confidence score and defaults to “Manual Review.”

1.4 Success Metrics

Metric	Target	Measurement
Precision	70%	True flagged RTOs / Total flagged orders
Recall	60%	True flagged RTOs / Actual RTOs
F1 Score	0.65	Harmonic mean of precision and recall
False Positive Rate	15%	Legit orders flagged / Total legit orders
Auto-Approval Rate	40%	Low-risk orders auto-shipped / Total orders
System Uptime	100%	Pipeline returns a response even if all APIs fail

2. System Architecture

2.1 High-Level Data Flow



2.2 State Management

SystemState (TypedDict) — single source of truth flowing through the pipeline:

```
SystemState = {
    # Input fields
    "order_id": str,
    "customer_id": str,
    "order_value": float,
    "category": str,
    "payment_method": str,
    "delivery_pincode": str,

    # Agent outputs
    "transaction_profile": TransactionProfile,
    "signal_data": SignalData,
    "risk_data": RiskData,
    "action_brief": ActionBrief,

    # Cross-cutting
    "company_name": str,          # Resolved merchant/customer name
    "confidence_score": float,    # 1.0 → 0.0
    "errors": Annotated[List[str], operator.add] # Append-only audit log
}
```

2.3 Design Decisions

Decision	Rationale
TypedDict over Pydantic for state	Lightweight, mutable, JSON-serializable; idiomatic for LangGraph
operator.add on errors	LangGraph reducer pattern —each agent appends, never overwrites
Sequential execution	Data dependencies: Signal Agent needs company_name from Profile Agent; Risk Agent needs both profiles; Synthesis needs all three
File-based cache (24h TTL)	Eliminates repeated DB/API calls; atomic writes with <code>os.replace()</code>
Deterministic risk scoring	Reproducible, auditable, no LLM hallucination on money decisions
Confidence scoring	Quantified reliability —merchants know when to trust auto-decisions

3. Agent Specifications

3.1 Agent 1: Transaction Profile Agent

File: agents/profile_agent.py

Purpose: Build a comprehensive risk profile from the customer's transaction history and current order attributes.

Inputs:

- order_id (from request)
- customer_id (from request)
- order_value (from request)
- category (from request)
- payment_method (from request)
- delivery_pincode (from request)

Outputs:

- transaction_profile (TransactionProfile dict)
- company_name (resolved customer/merchant name)
- Updated confidence_score
- Updated errors list

Data Source: Synthetic transaction database (synthetic_data/customers.csv, synthetic_data/orders.csv)

Execution Flow:

```
1. Check cache for customer profile (24hr TTL, key: "{customer_id}_profile")
2. If cache miss: Query synthetic database for customer history
3. Compute derived fields:
   - return_rate = returns / total_orders
   - avg_order_value = total_spent / total_orders
   - days_since_first_order = today - first_order_date
   - recent_returns_30d = count(returns in last 30 days)
   - pincode_rto_rate = historical RT0 rate for delivery_pincode
4. Assess data_available: all 6 critical fields present
5. Resolve company_name from customer record
6. Dock confidence:
   - Missing critical field → -0.4
   - Missing secondary field (pincode_rto_rate, account_age) → -0.05 each
```

Output Schema:

```
TransactionProfile = {
    "total_orders": int,           # Lifetime orders
    "return_rate": float,         # 0.0-1.0
    "avg_order_value": float,     # In INR
    "days_since_first_order": int,
    "recent_returns_30d": int,    # Returns in last 30 days
```

```

"order_value": float,          # Current order value
"category": str,              # fashion, electronics, home, beauty
"payment_method": str,        # cod, upi, card, wallet
"delivery_pincode": str,
"pincode_rto_rate": float,    # Historical RTO rate for pincode
"account_age_days": int,      # Days since first order
"data_available": bool,
}

```

Confidence Docking:

```

Base: 1.0
├─ Missing critical field (total_orders, return_rate, order_value) → -0.4
├─ Missing pincode_rto_rate (secondary) → -0.05
├─ Missing account_age_days (secondary) → -0.05
└─ Database timeout / corruption → -0.3
Minimum: 0.0 (clamped)

```

Fallback Behavior:

- Database timeout → use cached data if available; else mark `data_available=False`
- Invalid `customer_id` → error appended, `data_available=False`, confidence -0.4

3.2 Agent 2: Signal Agent

File: `agents/signal_agent.py`

Purpose: Gather external risk signals about the customer, delivery pincode, and product category.

Inputs:

- `company_name` (from Profile Agent)
- `customer_id` (from request)
- `delivery_pincode` (from request)
- `category` (from request)

Outputs:

- `signal_data` (SignalData dict)
- Updated `confidence_score`
- Updated errors list

Data Sources:

- **Primary:** Synthetic signal database (`synthetic_data/signals.csv`)
- **Secondary:** Mock external APIs (pincode RTO statistics, category return rates)
- **LLM:** Gemini Flash for narrative signal summarization (optional, fallback-heavy)

Execution Flow:

1. Build query from company_name + customer_id + pincode + category
2. Check cache for signals (24hr TTL, key: "{customer_id}_{pincode}_signals")
3. If cache miss: Query synthetic signal database
4. Gather signals:
 - Pincode RTO rate (from historical data)
 - Category return rate (from historical data)
 - Complaint score (0-1, from mock consumer forum data)
 - Social sentiment (-1 to 1, from mock social media scraping)
 - Recent events (list of strings: "High RTO pincode", "Product recall", etc.)
5. If LLM enabled: Send top signals to Gemini for structured JSON extraction
6. Schema validation: complaint_score $\in$ [0,1], sentiment $\in$ [-1,1]
7. If validation fails: 1 retry with stricter prompt
8. If both fail: degraded state (signals_available=False, defaults applied)

Output Schema:

```
SignalData = {  
    "pincode_rto_rate": float,          # 0.0-1.0, historical RTO for pincode  
    "category_return_rate": float,      # 0.0-1.0, historical return rate for category  
    "complaint_score": float,           # 0.0-1.0, customer complaint history  
    "social_sentiment": float,          # -1.0 to 1.0  
    "recent_events": List[str],         # ["High RTO pincode", "Festive season"]  
    "account_age_days": int,            # Redundant from profile, cross-validated  
    "signals_available": bool,  
}
```

Confidence Docking:

```
Base: 1.0 (inherited from Profile Agent)  
└─ No signals found for customer/pincode          → -0.1  
└─ Signal database timeout                        → -0.1  
└─ LLM validation failed (after retry)            → -0.1  
└─ Hostile signals detected (complaint_score > 0.7) → -0.05  
└─ Critical signal mismatch (profile says new, signals say old) → -0.1
```

Fail-Open Decisions:

- Missing pincode in signal database → use category average
- Missing complaint score → default to 0.0 (neutral)
- Missing social sentiment → default to 0.0 (neutral)

3.3 Agent 3: Risk Scorer Agent

File: agents/risk_agent.py

Purpose: Compute a deterministic 0–100 risk score from transaction profile and signal data.

Inputs:

- transaction_profile (from Profile Agent)

- signal_data (from Signal Agent)

Outputs:

- risk_data (RiskData dict)
- Updated errors list

Core Principle: Pure Python. Zero LLM involvement in scoring.

Deterministic Rules:

Condition	Weight	Risk Factor Text	Rationale
return_rate > 0.50	+30	“Serial returner (rate: {rate:.0%})”	Repeat behavior is strongest predictor
recent_returns_30d > 3	+25	“Return velocity spike ({count} in 30d)”	Recent acceleration matters
order_value > 10000 AND total_orders < 3	+20	“High-value new customer”	New + high value = high RTO risk
payment_method == "cod" AND order_value > 5000	+15	“COD high-value order”	COD has 2–3× RTO rate vs prepaid
pincode_rto_rate > 0.35	+15	“High-RTO delivery pincode ({rate:.0%})”	Geographic risk clustering
complaint_score > 0.70	+15	“High complaint history”	Customer satisfaction proxy
social_sentiment < -0.40	+10	“Negative social signals”	Brand/product dissatisfaction
account_age_days < 7	+15	“Brand new account”	No historical trust
category_return_rate > 0.30	+10	“High-RTO category ({rate:.0%})”	Fashion > electronics > home

Score Calculation:

```
risk_score = min(100.0, sum(weights of all triggered rules))
```

Recommendation Thresholds (deterministic, not LLM):

```
if risk_score <= 25.0:
    recommendation = "Auto-Approve"
elif risk_score <= 60.0:
    recommendation = "Manual Review"
else:
    recommendation = "Auto-Reject"
```

LLM Role (Narrative Only):

- Prompt includes: computed risk_score, risk_factors, transaction_profile, signal_data
- Explicit instruction: **“DO NOT RECOMPUTE scores or invent new quantitative flags. Write 2–3 sentences explaining the risk assessment to a merchant.”**
- Output: risk_narrative (string)

Fallback on LLM Failure:

- `risk_score` = **preserved from deterministic calculation** (NOT replaced)
- `risk_narrative` = “Risk assessment based on transaction history and delivery signals.”
- No confidence dock (risk is deterministic)

Output Schema:

```
RiskData = {  
    "risk_score": float,           # 0.0-100.0  
    "risk_factors": List[str],     # Human-readable triggered rules  
    "risk_narrative": str,         # LLM-generated explanation  
    "recommendation": str,         # Auto-Approve / Manual Review / Auto-Reject  
}
```

Boundary Testing Requirements:

- `return_rate` = 0.50 → does NOT trigger (boundary is exclusive)
- `return_rate` = 0.501 → triggers (+30)
- `order_value` = 10000 + `total_orders` = 3 → does NOT trigger (both conditions must be strict)
- `social_sentiment` = -0.40 → does NOT trigger (strictly less than)
- `social_sentiment` = -0.401 → triggers (+10)
- All None fields → never trigger (verified by tests)

3.4 Agent 4: Synthesis Agent

File: `agents/synthesis_agent.py`

Purpose: Generate the final merchant-facing action brief with a deterministic recommendation override.

Inputs:

- All prior agent outputs
- `confidence_score` (from state)
- `risk_data` (from Risk Agent)
- `transaction_profile` (from Profile Agent)
- `signal_data` (from Signal Agent)

Outputs:

- `action_brief` (ActionBrief dict)
- Updated errors list

Deterministic Recommendation Enforcement:

```
def _compute_recommendation(state):  
    confidence = state["confidence_score"]  
    risk_score = state["risk_data"]["risk_score"]
```

```

# Override: low confidence = always manual review
if confidence < 0.5:
    return "Manual Review"

# Use risk agent's recommendation
return state["risk_data"]["recommendation"]

```

LLM Role (Narrative Sections):

- Prompt requests 5 narrative sections (no recommendation field)
- LLM generates: `order_summary`, `risk_assessment`, `market_context`, `mitigation_suggestions`, `key_concerns`
- Python **overwrites** `recommended_action` with deterministic label
- Prevents LLM hallucination (e.g., “MUST SHIP IMMEDIATELY!!!”)

Fallback on LLM Failure:

```

{
    "order_summary": "Order analysis pending due to system error.",
    "risk_assessment": "Risk computed deterministically. Narrative unavailable.",
    "market_context": "System degradation detected.",
    "mitigation_suggestions": ["Review order manually."],
    "key_concerns": ["LLM synthesis pipeline error encountered."],
    "recommended_action": _compute_recommendation(state), # Still deterministic
}

```

Output Schema:

```

ActionBrief = {
    "order_summary": str,          # "Order #12345 for ₹8,500 (Fashion, COD)"
    "risk_assessment": str,        # "High risk: Serial returner + COD + High-RTO pincode"
    "market_context": str,         # "Festive season may increase RTO rates"
    "mitigation_suggestions": List[str], # ["Request partial prepayment", "Verify phone number"]
    "key_concerns": List[str],     # ["Return rate: 80%", "Pincode RTO: 42%"]
    "recommended_action": str,     # Auto-Approve / Manual Review / Auto-Reject
}

```

4. Cache Layer

File: `core/cache.py`

Design

- **File-based JSON cache** in `cache/` directory (gitignored)
- **24-hour TTL** (configurable via `TTL_SECONDS = 86400`)
- **Per-key storage**: `cache/{key}.json` (e.g., `cache/CUST_78901_profile.json`)
- **Atomic writes**: temp file → `fsync` → `os.replace()` (POSIX atomic rename)
- **File locking**: `fcntl` advisory locks on Unix; no-op on Windows
- **Corruption handling**: JSON decode error → delete file → return `None`

Cache Strategy

Operation	Behavior
Read	Check file mtime → if < 24hr → return data; else delete and return None
Write	Only cache successful responses (no error payloads)
Reject	Empty dicts, None values, error responses

Integration

```
# Profile Agent
cached = get_cached_response(f"{customer_id}_profile")
if cached: return cached, False # Cache hit

# ... database query ...

if valid_response:
    set_cached_response(f"{customer_id}_profile", data)
```

5. Configuration

File: core/config.py

Pydantic Settings

```
class Settings(BaseSettings):
    # API Keys
    GEMINI_API_KEY: str = Field(default="", validation_alias="GEMINI_API_KEY")

    # Model Configuration
    GEMINI_MODEL: str = "gemini-2.5-flash" # Verified working model

    # Cache Configuration
    CACHE_TTL_SECONDS: int = 86400 # 24 hours

    # Risk Thresholds
    SERIAL_RETURNER_THRESHOLD: float = 0.50
    HIGH_VALUE_THRESHOLD: float = 10000.0
    COD_HIGH_VALUE_THRESHOLD: float = 5000.0
    HIGH_RTO_PINCODE_THRESHOLD: float = 0.35
    HIGH_COMPLAINT_THRESHOLD: float = 0.70
    HOSTILE_SENTIMENT_THRESHOLD: float = -0.40
    NEW_ACCOUNT_DAYS: int = 7
    HIGH_CATEGORY_RTO_THRESHOLD: float = 0.30

    # Recommendation Thresholds
    AUTO_APPROVE_MAX_SCORE: float = 25.0
```

```
MANUAL_REVIEW_MAX_SCORE: float = 60.0
MIN_CONFIDENCE_FOR_AUTO: float = 0.50

# Data Paths
CUSTOMERS_CSV: str = "synthetic_data/customers.csv"
ORDERS_CSV: str = "synthetic_data/orders.csv"
SIGNALS_CSV: str = "synthetic_data/signals.csv"

model_config = SettingsConfigDict(env_file=".env", extra="ignore")
```

Environment Variables (.env.example)

```
# Required
GEMINI_API_KEY=your_gemini_key_here

# Optional
GEMINI_MODEL=gemini-2.5-flash
CACHE_TTL_SECONDS=86400
USE MOCK_DATA=false
```

6. API Layer

Files: main.py, api/routes.py

FastAPI Application

```
app = FastAPI(
    title="RTO Risk Scorer",
    version="0.1.0",
    description="Multi-agent AI pipeline for COD order RTO risk assessment"
)

# CORS: Restricted for production
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # TODO: Restrict in production
    allow_credentials=False,
    allow_methods=["GET", "POST"],
    allow_headers=["*"],
)
```

Endpoints

Method	Endpoint	Description	Auth
GET	/health	Container health probe	None
POST	/api/v1/analyze	Run full 4-agent pipeline	None
GET	/api/v1/metrics	Return precision/recall on held-out set	None

Table 6 – continued

Method	Endpoint	Description	Auth
GET	/api/v1/benchmark	Run full benchmark report	None
GET	/docs	Swagger UI (auto-generated)	None

Request/Response Models

AnalyzeRequest:

```
class AnalyzeRequest(BaseModel):
    order_id: str = Field(..., pattern=r"^[0-9]{6}$")
    customer_id: str = Field(..., pattern=r"^[CUST][0-9]{5}$")
    order_value: float = Field(..., gt=0)
    category: str = Field(..., pattern=r"^(fashion|electronics|home|beauty)$")
    payment_method: str = Field(..., pattern=r"^(cod|upi|card|wallet)$")
    delivery_pincode: str = Field(..., pattern=r"^[0-9]{6}$")
```

AnalyzeResponse:

```
class AnalyzeResponse(BaseModel):
    order_id: str
    customer_id: str
    risk_score: float
    recommendation: str
    confidence_score: float
    action_brief: ActionBrief
    audit_trail: List[str]
    processing_time_ms: int
```

7. Synthetic Data Layer

File: synthetic_data/generator.py

Customer Profiles

Generate **500 customers** with realistic RTO behavior patterns:

Customer Type	Weight	Behavior
Good	60%	Low return rate (0–10%), 5–50 orders
Occasional Returner	25%	Moderate return rate (10–30%), 3–30 orders
Serial Returner	10%	High return rate (50–90%), 10–40 orders
Fraudster	5%	Very high return rate (50–100%), 1–15 orders, often new accounts

Order Attributes

- **Order value:** 500– 25,000 (log-normal distribution)
- **Category:** fashion (40%), electronics (30%), home (20%), beauty (10%)
- **Payment method:** cod (60%), upi (20%), card (15%), wallet (5%)
- **Pincode:** 20 pincodes with varying RTO rates (0.10–0.50)
- **Ground truth label:** legit or rto (based on customer type + noise)

Signal Data

- **Pincode RTO rate:** Historical RTO for pincode (0.10–0.50)
- **Category return rate:** Historical return rate for category (0.15–0.40)
- **Complaint score:** 0.0–1.0 (higher for fraudsters)
- **Social sentiment:** -1.0 to 1.0 (lower for fraudsters)
- **Recent events:** Random selection from [“Festive season”, “High RTO pincode”, “None”]

Data Split

- **Training set:** 400 customers/orders (for threshold tuning)
- **Held-out test set:** 100 customers/orders (for final metrics)
- **Ground truth:** Used only for evaluation, never fed to agents

8. Evaluation Layer

File: evaluation/metrics.py

Metrics Computed

Metric	Formula	Target
True Positives (TP)	Flagged as RTO + Actually RTO	—
False Positives (FP)	Flagged as RTO + Actually legit	—
False Negatives (FN)	Not flagged + Actually RTO	—
True Negatives (TN)	Not flagged + Actually legit	—
Precision	$TP / (TP + FP)$	70%
Recall	$TP / (TP + FN)$	60%
F1 Score	$2 \times (Precision \times Recall) / (Precision + Recall)$	0.65
False Positive Rate	$FP / (FP + TN)$	15%
Auto-Approval Rate	$Score_{25} / Total$	40%
Money Saved	$(Prevented\ RTOs \times 100\ avg\ cost) - (FP \times 50\ review\ cost)$	> 0

Benchmark Report

```
{
  "total_orders": 100,
  "true_positives": 18,
  "false_positives": 7,
```

```
"false_negatives": 12,
"true_negatives": 63,
"precision": 0.72,
"recall": 0.60,
"f1_score": 0.655,
"false_positive_rate": 0.10,
"auto_approval_rate": 0.45,
"estimated_money_saved_inr": 1250.00
}
```

9. Testing Strategy

Test Files

File	Tests	Focus
tests/test_profile_agent.py	15	Cache, DB fallback, edge cases, missing fields
tests/test_signal_agent.py	20	Signal parsing, schema validation, retry, hostile signals
tests/test_risk_agent.py	18	Deterministic rules (boundaries, None-safety), LLM fallback
tests/ test_synthesis_agent.py	8	Label logic, programmatic override, malformed JSON
tests/test_evaluation.py	10	Metrics computation, boundary conditions
tests/ test_pipeline_integration.py	12	E2E mock pipeline, recommendation logic, state preservation
Total	83	—

Key Test Patterns

- **Boundary testing:** Exact thresholds (0.50, 10000, 5000, 0.35, 0.70, -0.40)
- **None-safety:** Missing data doesn't crash or trigger false flags
- **Error accumulation:** Prior errors preserved through pipeline
- **LLM override:** Hallucinated labels corrected to deterministic values
- **Cache behavior:** Hit/miss, corruption handling, TTL expiry
- **Confidence docking:** Cumulative docks, clamping to [0, 1]

10. Frontend (Minimal)

Files: frontend/index.html, frontend/app.js, frontend/styles.css

Design

Simple single-page dashboard:

- **Input form:** Order ID, Customer ID, Order Value, Category, Payment Method, Pincode
- **Submit button:** Calls `/api/v1/analyze`
- **Result card:** Risk score (color-coded: green 25, yellow 26-60, red >60)
- **Recommendation badge:** Auto-Approve / Manual Review / Auto-Reject
- **Brief sections:** Expandable cards for each section of ActionBrief
- **Audit trail:** Collapsible list of errors and decisions
- **Confidence indicator:** Progress bar showing confidence score

Technology

- Pure HTML/CSS/JS (no framework — fast to build, easy to demo)
- Fetches API via `fetch()`
- Responsive design (works on mobile for demo)

11. Deployment

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY pyproject.toml .
RUN pip install --no-cache-dir -e "."

COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

docker-compose.yml

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
    volumes:
      - ./cache:/app/cache
      - ./synthetic_data:/app/synthetic_data
```

CI/CD (GitHub Actions)

```
name: Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - run: pip install -e ".[dev]"
      - run: pytest tests/ -v --cov
```

12. Project Roadmap

Phase 1: Foundation (Days 1–2)

Day	Task	Deliverable
Day 1 (Aug 26)	Set up repo, copy Chrimatos core, fix critical bugs	Working skeleton
Day 2 (Aug 27)	Build synthetic data generator + 500 records	customers.csv, orders.csv, signals.csv

Phase 2: Agents (Days 3–6)

Day	Task	Deliverable
Day 3 (Aug 28)	Build Profile Agent + tests	Agent 1 working, 15 tests passing
Day 4 (Aug 29)	Build Signal Agent + tests	Agent 2 working, 20 tests passing
Day 5 (Aug 30)	Build Risk Agent (deterministic rules) + tests	Agent 3 working, 18 tests passing
Day 6 (Aug 31)	Build Synthesis Agent + evaluation metrics	Agent 4 working, metrics computed

Phase 3: Integration (Days 7–8)

Day	Task	Deliverable
Day 7 (Sep 1)	Full pipeline integration + API layer + frontend	End-to-end demo working

Table 12 – continued

Day	Task	Deliverable
Day 8 (Sep 2)	Docker + deployment + README polish	<code>docker-compose</code> up works

Phase 4: Polish (Days 9–10)

Day	Task	Deliverable
Day 9 (Sep 3)	Pitch video recording + editing	5-min video
Day 10 (Sep 4)	Buffer: bug fixes, metric improvements, edge cases	Submission-ready
Sep 5	SUBMIT	Public repo + video + architecture docs

13. File Structure Commands

```
# Create project root
mkdir rto-risk-scorer && cd rto-risk-scorer

# Initialize git
git init

# Create directory structure
mkdir -p agents core synthetic_data evaluation api tests frontend cache

# Create empty __init__.py files
touch agents/__init__.py
touch core/__init__.py
touch api/__init__.py
touch tests/__init__.py

# Create core files
touch core/state.py
touch core/orchestrator.py
touch core/cache.py
touch core/config.py
touch core/clients.py

# Create agent files
touch agents/profile_agent.py
touch agents/signal_agent.py
touch agents/risk_agent.py
touch agents/synthesis_agent.py

# Create API files
touch api/routes.py
```

```

# Create data/evaluation files
touch synthetic_data/generator.py
touch synthetic_data/customers.csv
touch synthetic_data/orders.csv
touch synthetic_data/signals.csv
touch evaluation/metrics.py
touch evaluation/benchmark.py

# Create test files
touch tests/test_profile_agent.py
touch tests/test_signal_agent.py
touch tests/test_risk_agent.py
touch tests/test_synthesis_agent.py
touch tests/test_evaluation.py
touch tests/test_pipeline_integration.py

# Create frontend files
touch frontend/index.html
touch frontend/app.js
touch frontend/styles.css

# Create root files
touch main.py
touch pyproject.toml
touch pytest.ini
touch Dockerfile
touch docker-compose.yml
touch .env.example
touch .gitignore
touch README.md

# Verify structure
find . -type f | sort

```

Final Directory Tree

```

rto-risk-scorer/
├── agents/
│   ├── __init__.py
│   ├── profile_agent.py          # Agent 1: Transaction Profile
│   ├── signal_agent.py          # Agent 2: External Signals
│   ├── risk_agent.py            # Agent 3: Deterministic Scorer
│   └── synthesis_agent.py       # Agent 4: Action Brief
├── core/
│   ├── __init__.py
│   ├── state.py                 # SystemState, TypedDicts, init_state()
│   ├── orchestrator.py         # LangGraph StateGraph compilation
│   ├── cache.py                # File cache (24h TTL, atomic writes, locking)
│   ├── config.py               # Pydantic Settings (env-driven)
│   └── clients.py              # Singleton clients (httpx, Gemini)

```

```

├─ api/
│   ├── __init__.py
│   └── routes.py                # FastAPI endpoints (/analyze, /metrics, /health)
├─ synthetic_data/
│   ├── generator.py            # Synthetic data generation
│   ├── customers.csv           # Generated customer profiles
│   ├── orders.csv              # Generated orders
│   └── signals.csv             # Generated external signals
├─ evaluation/
│   ├── metrics.py              # Precision, recall, F1, FPR
│   └── benchmark.py            # Full benchmark runner
├─ tests/
│   ├── __init__.py
│   ├── test_profile_agent.py    # 15 tests
│   ├── test_signal_agent.py     # 20 tests
│   ├── test_risk_agent.py       # 18 tests
│   ├── test_synthesis_agent.py  # 8 tests
│   ├── test_evaluation.py       # 10 tests
│   └── test_pipeline_integration.py # 12 tests
├─ frontend/
│   ├── index.html              # Dashboard HTML
│   ├── app.js                  # Dashboard JS
│   └── styles.css               # Dashboard CSS
├─ cache/                       # Local cache (gitignored)
├─ main.py                      # FastAPI app entry
├─ pyproject.toml               # Dependencies + project metadata
├─ pytest.ini                   # Pytest configuration
├─ Dockerfile                   # Container definition
├─ docker-compose.yml           # Local orchestration
├─ .env.example                 # Environment variable template
├─ .gitignore                   # Git ignore rules
└─ README.md                    # Project documentation

```

14. Pitch Video Script (5 Minutes)

0:00–0:30 — The Problem

“In Indian e-commerce, 25–40% of COD orders never reach the customer. They RTO — Return to Origin. Each RTO costs the merchant 50–150. For a merchant doing 10 lakh/month, that’s 2.5–4 lakh in preventable losses.”

0:30–1:30 — The Demo

“Here’s a COD order for 12,000 from a new customer in a high-RTO pincode. My system scores it 85 — Auto-Reject. Here’s a prepaid UPI order for 1,500 from a loyal customer — score 12, Auto-Approve. Here’s the edge case — missing pincode data, confidence drops to 0.6, defaults to Manual Review.”

1:30–2:30 — The Architecture

“Four agents in a LangGraph pipeline. The first builds a transaction profile. The second gathers external signals — pincode RTO rates, complaint scores. The third computes risk using pure Python rules — no LLM black box. The fourth generates a merchant brief. The LLM only writes explanations, never changes the score.”

2:30–3:30 — The Metrics

“On 100 held-out orders: 72% precision, 60% recall, F1 of 0.66. False positive rate is 10%. 45% of orders get auto-approved, saving review time. Estimated money saved: 1,250 per 100 orders.”

3:30–4:30 — Failure Handling

“If the database is down, cached data keeps working. If the LLM fails, deterministic rules still produce a score. If confidence is below 0.5, we always route to manual review. Here’s a run with everything failing — it still returns a safe default.”

4:30–5:00 — Honest Limitations & Next Steps

“This uses synthetic data. Production needs integration with Razorpay’s transaction APIs and real merchant history. The architecture is built for that — swap the synthetic data source, same pipeline. Next: A/B test with a live merchant, tune thresholds based on actual RTO rates.”

15. Risk & Mitigation

Risk	Impact	Mitigation
Synthetic data not realistic	Metrics misleading	Use realistic distributions, validate with industry benchmarks
Gemini API quota exhausted	LLM narratives fail	Deterministic fallback always works; no API dependency for scoring
10 days insufficient	Incomplete submission	Prioritize: agents > metrics > frontend > video. Cut frontend if needed.
Evaluation questions on real data	Credibility hit	Be honest: “Synthetic data, ready for real integration.”
CORS/security flags	Technical rejection	Fix CORS, add input validation, no exposed secrets

16. Appendix: Chrimatos Reuse Mapping

Chrimatos Component	RTO Risk Scorer Equivalent	Reuse %
core/state.py	Same pattern, renamed fields	90%
core/orchestrator.py	Same LangGraph DAG	95%
core/cache.py	Same file cache	100%
core/config.py	Add new thresholds	80%

Table 15 – continued

Chrimatos Component	RTO Risk Scorer Equivalent	Reuse %
core/clients.py	Same singleton pattern	100%
agents/financial_agent.py	agents/profile_agent.py	70%
agents/news_agent.py	agents/signal_agent.py	70%
agents/risk_agent.py	agents/risk_agent.py	85%
agents/synthesis_agent.py	agents/synthesis_agent.py	85%
api/routes.py	Adapt endpoints	60%
Tests	Same patterns, new data	75%
Overall	—	~80%

End of PRD